

Informe de Implementación — Instrucciones RVC para la Entrega 3

Informe de Implementación — Instrucciones RVC para la Entrega 3.....	1
4. Implementación de instrucciones.....	2
4.1 Instrucciones de carga y almacenamiento en el Cuadrante 0.....	2
4.1.1 C.LW (Load Word comprimido).....	3
4.1.2 C.SW (Store Word comprimido).....	6
4.2 Instrucciones de carga y almacenamiento relativas al stack pointer.....	8
4.2.1 C.LWSP (Load Word comprimido con stack pointer).....	8
4.2.2 C.SWSP (Store Word comprimido con stack pointer).....	10
4.3 Instrucciones de salto condicional (Branches) en el Cuadrante 1.....	12
4.3.1 C.BEQZ (Branch if Equal to Zero comprimido).....	12
4.3.2 C.BNEZ (Branch if Not Equal to Zero comprimido).....	15
4.4 Instrucciones de salto incondicional.....	16
4.4.1 C.J (Jump comprimido).....	17
4.4.2 C.JAL (Jump and Link comprimido).....	18
4.4.3 C.JR (Jump Register comprimido).....	20
4.4.4 C.JALR (Jump and Link Register comprimido).....	22
4.5 Cambios en el datapath para adaptación.....	23

En el presente documento se aborda la implementación de diez instrucciones comprimidas de la extensión RVC, correspondientes a los cuadrantes Q0, Q1 y Q2 del repertorio de 16 bits de RISC-V. Para cada instrucción, se ha elaborado una explicación de su funcionamiento semántico, el detalle de su codificación de 16 bits, el análisis del código Verilog que realiza la expansión a 32 bits y, cuando corresponda, una discusión sobre las implicaciones en el datapath del pipeline. Se prestará especial atención a la trazabilidad entre la instrucción comprimida, su expansión en el descompresor y las señales de control que genera el decodificador principal, de manera que quede clara la interacción entre los distintos módulos del procesador.

4. Implementación de instrucciones

4.1 Instrucciones de carga y almacenamiento en el Cuadrante 0

El cuadrante Q0 de la extensión RVC agrupa las instrucciones de acceso a memoria que utilizan exclusivamente registros del rango comprimido x8 a x15. Estas instrucciones se identifican porque los dos bits menos significativos del halfword toman el valor 2'b00. Dentro de este cuadrante, el campo funct3 ubicado en los bits 15 a 13 distingue la operación específica. Las instrucciones C.LW y C.SW, que corresponden a los valores 010 y 110 de dicho campo, son las primeras en ser analizadas.

4.1.1 C.LW (Load Word comprimido)

La instrucción C.LW carga una palabra de 32 bits desde la memoria de datos hacia un registro destino del rango comprimido. Su sintaxis ensamblador es `c.lw rd', offset(rs1')`, donde tanto `rd'` como `rs1'` pertenecen al conjunto de registros x8 a x15. La dirección de memoria se obtiene sumando el valor del registro base `rs1'` con un desplazamiento de 7 bits, cuyo valor se encuentra implícitamente multiplicado por 2 (alineación a halfword). Esta instrucción constituye la versión compacta de la instrucción estándar `lw rd, offset(rs1)` del repertorio RV32I, y su propósito principal es reducir el tamaño del código en programas donde los accesos a memoria utilizan registros del rango bajo.

La codificación de la instrucción en su formato de 16 bits se presenta en la tabla siguiente. En ella, el lector puede observar la distribución de los campos dentro del halfword, donde los bits de desplazamiento aparecen dispersos en cuatro grupos distintos.

15–13	12	11–10	9–7	6	5	4–2	1–0
010	offset[5]	offset[4 :3]	rs1'	offset[6]	offset[2]	rd'	00

Los campos rd' y rs1' ocupan 3 bits cada uno y codifican los registros del rango x8 a x15 mediante la correspondencia {2'b01, campo_3bits}. El desplazamiento se compone de 7 bits distribuidos en las posiciones indicadas, donde los bits offset[1:0] se asumen cero. De esta manera, el offset efectivo resulta de la concatenación {offset[6], offset[5], offset[4:3], offset[2], 2'b00}, lo que produce un valor múltiplo de 4 con un rango de hasta 124 bytes.

El fragmento del descompresor que realiza la expansión de esta instrucción se encuentra en las líneas 16 a 19 del archivo descompresor.v. El código se reproduce a continuación.

```

3'b010: begin
    instr32 = {5'b00000, instr16[5], instr16[12:10], instr16[6], 2'b00,
              rs1_p, 3'b010, rd_p, 7'b00000011};
end

```

La expansión reconstruye una instrucción de formato I-type cuyo opcode es 0000011, que corresponde a la operación LOAD en el repertorio RV32I. El campo funct3 toma el valor 010 para indicar que la carga es de tipo word (32 bits). El registro fuente rs1 se obtiene del cable rs1_p, definido como {2'b01, instr16[9:7]}, mientras que el registro destino rd proviene del cable rd_p, definido

como {2'b01, instr16[4:2]}. La concatenación {2'b01, ...} convierte el campo de 3 bits de la instrucción comprimida en un número de registro de 5 bits comprendido entre 8 y 15.

El inmediato de 12 bits se construye a partir de los bits dispersos del offset. Los cinco bits más significativos se fijan a cero (5'b00000), lo que limita el rango del inmediato a valores no negativos menores que 2048, consistente con la especificación RVC. A continuación se coloca instr16[5], que corresponde al bit offset[2] de la codificación. Los bits instr16[12:10] proporcionan los valores offset[5:3]. El bit instr16[6] entrega offset[6]. Finalmente, los dos bits menos significativos del inmediato se fuerzan a 2'b00, preservando la alineación a 4 bytes que exige la instrucción lw de RV32I.

Una vez que la instrucción expandida abandona el descompresor y atraviesa el registro IF/ID, ingresa a la etapa de decodificación donde el módulo maindec.v la reconoce. El lector puede verificar en la línea 15 de maindec.v que el opcode 0000011 produce las siguientes señales de control:

```
7'b0000011: controls = 13'b1_000_1_0_01_0_00_0_0;
```

Estas señales indican que RegWrite = 1 (se escribe el registro destino), ALUSrc = 1 (el segundo operando de la ALU es el inmediato extendido), ImmSrc = 000 (extensión de inmediato tipo I), ResultSrc = 01 (el resultado proviene de la memoria de datos), y MemWrite = 0 (no hay escritura en memoria). La extensión del inmediato I-type se realiza en el módulo extend.v, específicamente en la línea 7:

```
3'b000: immext_reg = {{20{instr[31]}}, instr[31:20]};
```

Este bloque toma el bit más significativo del campo inmediato (instr[31]) y lo extiende con signo 20 posiciones hacia la izquierda, produciendo un valor de 32

bits que la ALU suma con el contenido de rs1 para calcular la dirección efectiva. En la etapa EX, la ALU recibe la señal ALUControl = 0000 (ADD) desde el decodificador aludec.v, dado que ALUOp = 00 para las instrucciones de carga. En cuanto al hazard unit, ubicado en hazard_unit.v, es importante destacar que opera sin modificaciones. La señal MemReadE se activa porque ResultSrcE[0] = 1 (tal como se define en la línea 36 de pipeline.v: wire MemReadE = ResultSrcE[0]). Si la instrucción siguiente en el pipeline intenta leer el registro rd de C.LW como fuente, el hazard unit detecta la dependencia y genera un stall de un ciclo, congelando las etapas IF e ID mientras la etapa MEM completa la lectura. Esta detección funciona correctamente porque el hazard unit recibe los números de registro RdE, Rs1D y Rs2D de la instrucción ya expandida de 32 bits, donde rd_p y rs1_p son valores de 5 bits perfectamente válidos en el rango 8 a 15.

4.1.2 C.SW (Store Word comprimido)

La instrucción C.SW almacena una palabra de 32 bits desde un registro fuente del rango comprimido hacia la memoria de datos. Su sintaxis es c.sw rs2', offset(rs1'), donde rs2' contiene el dato que se desea escribir y rs1' proporciona la dirección base. El offset, al igual que en C.LW, es un valor de 7 bits con alineación a 2 bytes. Esta instrucción es la versión compacta de sw rs2, offset(rs1) de RV32I.

La codificación de 16 bits de C.SW utiliza el mismo cuadrante y el campo funct3 = 110. La distribución de sus campos se muestra en la tabla que sigue.

15–13	12	11–10	9–7	6	5	4–2	1–0
110	offset[5]	offset[4:3]	rs1'	offset[6]	offset[2]	rs2'	00

El lector notará que la distribución del offset es idéntica a la de C.LW, lo cual es consistente con la especificación RISC-V. La diferencia radica en que el campo

de 3 bits en las posiciones 4 a 2 ahora codifica el registro fuente rs2' en lugar del registro destino. El registro base rs1' se mantiene en las posiciones 9 a 7. El código de expansión en el descompresor corresponde a las líneas 20 a 25 del archivo decompressor.v.

```
3'b110: begin
    instr32 = {5'b00000, instr16[5], instr16[12],
               rs2_p, rs1_p, 3'b010,
               instr16[11:10], instr16[6], 2'b00,
               7'b0100011};
end
```

Esta expansión reconstruye una instrucción de formato S-type con opcode 0100011, que corresponde a la operación STORE en RV32I. El campo funct3 se fija en 010 para indicar almacenamiento de palabra completa. El registro rs1 (dirección base) se obtiene de rs1_p ({2'b01, instr16[9:7]}) y el registro rs2 (dato a escribir) se obtiene de rs2_p ({2'b01, instr16[4:2]}).

La construcción del inmediato S-type merece atención especial, pues el formato S-type de RV32I divide el inmediato de 12 bits en dos partes: los bits imm[11:5] se colocan en el campo que ocupa las posiciones 31 a 25 de la instrucción, y los bits imm[4:0] se colocan en las posiciones 11 a 7. La expansión debe, por lo tanto, distribuir correctamente los 7 bits del offset RVC en estas dos porciones. Los bits imm[11:5] de la instrucción S-type se forman con {5'b00000, instr16[5], instr16[12]}. Los bits imm[4:0] se forman con {instr16[11:10], instr16[6], 2'b00}. Es importante destacar que el bit instr16[5] corresponde a offset[2] de la codificación RVC, pero en la expansión S-type aparece en la posición imm[6]. De manera similar, instr16[12] es offset[5] en RVC y pasa a ser imm[5] en el formato S-type. Esta reorganización de bits es necesaria para cumplir con la disposición del campo inmediato que espera la instrucción sw de 32 bits.

Cuando la instrucción expandida llega a la etapa ID, el módulo maindec.v la decodifica mediante la línea 16:

```
7'b0100011: controls = 13'b0_001_1_1_00_0_00_0_0;
```

Las señales generadas son RegWrite = 0 (no se escribe en el banco de registros), ALUSrc = 1 (el segundo operando de la ALU es el inmediato), ImmSrc = 001 (extensión de inmediato tipo S), MemWrite = 1 (escritura en memoria), ResultSrc = 00 y ALUOp = 00. La extensión del inmediato S-type se realiza en extend.v mediante la línea 8:

```
3'b001: immext_reg = {{20{instr[31]}}, instr[31:25], instr[11:7]};
```

Este bloque reconstruye el inmediato S-type tomando los bits instr[31:25] (bits 11 a 5 del inmediato) y instr[11:7] (bits 4 a 0), y extendiendo con signo a partir del bit instr[31]. La ALU suma este valor con el contenido de rs1 para calcular la dirección donde se escribirá el dato. En la etapa MEM, el módulo dmem.v recibe la señal MemWrite = 1 y ejecuta la escritura en la dirección calculada.

El hazard unit, de manera análoga al caso de C.LW, no requiere modificaciones. Aunque C.SW no produce un resultado que deba ser adelantado (pues RegWrite = 0), el hazard unit sí debe detectar si rs1' o rs2' dependen de una instrucción anterior en el pipeline. Dado que los números de registro que recibe son los mismos que produciría una instrucción sw estándar de RV32I, la lógica de forwarding y stalling existente maneja correctamente estas dependencias sin necesidad de adaptación.

4.2 Instrucciones de carga y almacenamiento relativas al stack pointer

El cuadrante Q2 de la extensión RVC, identificado por los bits [1:0] = 2'b10, contiene instrucciones que operan con el registro x2 (stack pointer) como base de direccionamiento fija. A diferencia de las instrucciones del cuadrante Q0,

estas no utilizan registros comprimidos de 3 bits para la base, sino que hardcodean el valor $rs1 = x2$ (5'b00010). El registro destino o fuente puede ser cualquier registro completo de 5 bits del banco ($x0$ a $x31$), lo que las hace más flexibles que sus contrapartes del cuadrante Q0. Dentro de este cuadrante, las instrucciones C.LWSP y C.SWSP se encargan de la carga y el almacenamiento de palabras utilizando el stack pointer como dirección base.

4.2.1 C.LWSP (Load Word comprimido con stack pointer)

La instrucción C.LWSP carga una palabra de 32 bits desde la memoria hacia un registro destino, utilizando el stack pointer ($x2$) como dirección base. Su sintaxis es `c.lwsp rd, offset(x2)`, donde `rd` puede ser cualquier registro del banco a excepción de $x0$. El `offset`, a diferencia de C.LW, tiene 8 bits con alineación a 4 bytes, lo que proporciona un rango de direccionamiento mayor. Esta instrucción se expande a `lw rd, offset(x2)`.

La codificación de 16 bits de C.LWSP se muestra en la tabla siguiente.

15–13	12	11–7	6–4	3–2	1–0
010	offset[4]	rd	offset[5]	offset[7:6]	10

El registro destino `rd` ocupa 5 bits completos en las posiciones 11 a 7, lo que permite direccionar cualquier registro del banco. El `offset` de 8 bits se distribuye en tres grupos: `offset[7:6]` en los bits 3 a 2, `offset[5]` en los bits 6 a 4, y `offset[4]` en el bit 12. Los bits `offset[3:0]` se asumen cero, lo que produce un `offset` efectivo múltiplo de 16. El `offset` real se obtiene como `{offset[7:4], 4'b0000}`, con un rango de 0 a 496 bytes.

El código de expansión se encuentra en las líneas 109 a 112 del archivo `decompressor.v`.

```
3'b010: begin
```



```

instr32 = {4'b0000, instr16[3:2], instr16[12], instr16[6:4], 2'b00,
          5'b00010, 3'b010, rd_f, 7'b0000011};
end

```

La expansión produce una instrucción lw con formato I-type. El opcode 0000011 y el funct3 = 010 son los mismos que en C.LW, lo cual era de esperarse pues ambas instrucciones realizan una carga de palabra. La diferencia fundamental radica en dos aspectos. Primero, el registro base rs1 ya no proviene de un campo de 3 bits, sino que se fija directamente a 5'b00010, que corresponde al registro x2 (stack pointer). Segundo, el registro destino rd se obtiene del cable rd_f, definido como instr16[11:7], que proporciona 5 bits completos sin ningún prefijo adicional, permitiendo direccionar cualquier registro.

La construcción del inmediato I-type sigue una lógica similar a la de C.LW, pero con una disposición de bits diferente. Los cuatro bits más significativos se fijan a cero (4'b0000). Luego se colocan instr16[3:2], que corresponden a offset[7:6]. A continuación se inserta instr16[12], que es offset[4]. Siguen instr16[6:4], que representan offset[5]. Finalmente, los dos bits menos significativos se fuerzan a 2'b00. El inmediato I-type resultante es {4'b0000, offset[7:6], offset[4], offset[5], 2'b00}.

Es importante señalar que la señal ImmSrc = 000 que genera maindec.v para el opcode 0000011 produce una extensión de inmediato I-type estándar en extend.v. El hecho de que el campo inmediato tenga ceros en los bits superiores no afecta el resultado de la extensión con signo, pues el bit instr[31] es cero en todos los casos (el offset RVC está en el rango 0 a 496), por lo que la extensión produce un valor positivo.

Respecto al hazard unit, la situación es idéntica a la de C.LW: la señal MemReadE se activa indicando una lectura en memoria, y si la instrucción siguiente utiliza el registro rd de C.LWSP como fuente, se genera un stall de un ciclo. Dado que rd proviene de rd_f con 5 bits completos, cualquier registro del banco puede ser detectado como destino de la dependencia.

4.2.2 C.SWSP (Store Word comprimido con stack pointer)

La instrucción C.SWSP almacena una palabra de 32 bits desde un registro fuente hacia la memoria, utilizando el stack pointer (x2) como dirección base. Su sintaxis es `c.swsp rs2, offset(x2)`, donde rs2 puede ser cualquier registro del banco de registros. Esta instrucción es la versión compacta de `sw rs2, offset(x2)` y, junto con C.LWSP, permite gestionar el frame de la pila de manera eficiente en términos de tamaño de código.

El formato de 16 bits de C.SWSP se presenta en la tabla siguiente.

15–13	12	11–7	6–2	1–0
110	offset[2]	rs2	offset[5:3]	10

A diferencia de C.SW del cuadrante Q0, esta instrucción utiliza un campo completo de 5 bits para el registro fuente rs2 (posiciones 11 a 7), permitiendo almacenar cualquier registro. El offset de 8 bits se distribuye en dos grupos: offset[5:3] en los bits 6 a 2 y offset[2] en el bit 12. Los bits offset[7:6] se almacenan, pero en una posición distinta que deberá ser reinterpretada en la expansión. El offset efectivo, con alineación a 4 bytes, resulta de {offset[7:2], 2'b00}.

El código de expansión se encuentra en las líneas 130 a 135 del archivo decompressor.v.

```
3'b110: begin
    instr32 = {4'b0000, instr16[8:7], instr16[12],
               rs2_f, 5'b00010, 3'b010,
               instr16[11:9], 2'b00,
               7'b0100011};
end
```

La expansión produce una instrucción S-type con opcode 0100011. El registro base rs1 se fija a 5'b00010 (x2), hardcodedo el stack pointer como base. El registro fuente rs2 se obtiene del cable rs2_f, definido como instr16[6:2] en la línea 11 del descompresor. Nótese que el campo rs2 en la instrucción de 16 bits ocupa las posiciones 11 a 7, pero el cable rs2_f se define como instr16[6:2]. Esto se debe a que en el formato CSS (Stack-relative Store) de RVC, el registro fuente se codifica en los bits 6 a 2, no en los bits 11 a 7. La línea 130 del descompresor utiliza rs2_f correctamente, tomando el campo de 5 bits de las posiciones 6 a 2.

La construcción del inmediato S-type en C.SWSP sigue un patrón específico. Los bits imm[11:5] se forman con {4'b0000, instr16[8:7], instr16[12]}. Aquí, instr16[8:7] corresponde a offset[7:6] de la codificación RVC e instr16[12] corresponde a offset[2]. Los bits imm[4:0] se forman con {instr16[11:9], 2'b00}, donde instr16[11:9] representa offset[5:3]. Esta disposición, aunque diferente de la de C.SW, también es válida para el formato S-type y produce el offset correcto en la extensión de inmediatos.

Al igual que C.SW, esta instrucción es decodificada por maindec.v en la línea 16 con el mismo conjunto de señales de control. La etapa EX suma x2 con el inmediato extendido para obtener la dirección, y la etapa MEM escribe el contenido de rs2_f en dicha dirección. El hazard unit maneja las dependencias sobre x2 y sobre rs2_f de manera transparente.

4.3 Instrucciones de salto condicional (Branches) en el Cuadrante 1

El cuadrante Q1 de la extensión RVC, identificado por los bits [1:0] = 2'b01, contiene las instrucciones de salto condicional comprimidas, entre otras. Dentro de este cuadrante, los campos funct3 = 110 y funct3 = 111 corresponden a C.BEQZ y C.BNEZ respectivamente. Ambas instrucciones comparan un registro del rango comprimido con cero y, si la condición se cumple, transfieren el control a una dirección objetivo calculada como PC + offset. La diferencia fundamental entre ambas radica en la condición de comparación: igualdad a cero para C.BEQZ y desigualdad a cero para C.BNEZ.

4.3.1 C.BEQZ (Branch if Equal to Zero comprimido)

La instrucción C.BEQZ evalúa si el contenido de un registro del rango comprimido es igual a cero. En caso afirmativo, el flujo del programa se desvía a la dirección PC + offset, donde el offset es un valor de 9 bits con signo y alineación a 2 bytes. Si la condición no se cumple, la ejecución continúa secuencialmente con la instrucción siguiente. La sintaxis de la instrucción es c.beqz rs1', offset. Su equivalente en RV32I es beq rs1', x0, offset, donde el segundo operando se fija al registro x0 (siempre cero).

La codificación de 16 bits de C.BEQZ utiliza un formato escalonado para el offset, como se muestra en la tabla siguiente.

15–13	12	11–10	9–7	6–5	4–3	2	1–0
110	offset[5]	offset[2 :1]	rs1'	offset[4 :3]	offset[7 :6]	offset[8]	01

El offset de 9 bits se encuentra distribuido en seis grupos distintos dentro del halfword. El orden de los bits para reconstruir el offset B-type de 13 bits (con el bit offset[0] implícitamente cero) es: {offset[8], offset[4:3], offset[5], offset[2:1], offset[7:6], 1'b0}. Esta dispersión es característica de las instrucciones branch comprimidas y obedece a la necesidad de compartir la misma disposición de campos que el formato B-type de RV32I para simplificar la expansión.

El código de expansión en el descompresor se encuentra en las líneas 84 a 91 del archivo decompressor.v.

```
3'b110: begin
```

```
    instr32 = {instr16[12],  
               instr16[12], instr16[12], instr16[12],  
               instr16[6], instr16[5], instr16[2],
```

```

        5'b00000, rs1_p, 3'b000,
        instr16[11:10], instr16[4:3], instr16[12],
        7'b1100011};
end

```

La expansión reconstruye una instrucción B-type con opcode 1100011 y funct3 = 000. El valor funct3 = 000 es crucial, pues corresponde a la condición BEQ (branch if equal) en el repertorio RV32I. El registro rs1 se obtiene de rs1_p ({2'b01, instr16[9:7]}), mientras que rs2 se fija a 5'b00000 (x0), lo que hace que la comparación sea siempre contra cero.

La construcción del inmediato B-type merece un análisis detallado, pues es uno de los formatos más complejos de RV32I. El formato B-type distribuye el offset de 13 bits (bits 0 a 12, con el bit 0 implícito en cero) en cuatro grupos dentro de la instrucción de 32 bits: imm[12] en la posición 31, imm[10:5] en las posiciones 30 a 25, imm[4:1] en las posiciones 11 a 8, y imm[11] en la posición 7. La expansión debe, por lo tanto, reorganizar los 9 bits del offset RVC en estas cuatro porciones.

Los bits imm[12] y imm[10:5] de la instrucción B-type constituyen la porción superior del inmediato. En la expansión, estos bits se forman mediante la concatenación {instr16[12], instr16[12], instr16[12], instr16[12], instr16[6], instr16[5], instr16[2]}. Aquí, instr16[12] es offset[5] de la codificación RVC, pero se repite cuatro veces para ocupar los bits imm[12:9]. Esto es posible porque la especificación RVC garantiza que offset[8:5] tiene sus bits superiores iguales al bit de signo cuando el offset es pequeño. Los bits instr16[6], instr16[5] e instr16[2] corresponden a offset[4], offset[3] y offset[8] respectivamente, pero la expansión los recoloca como imm[8], imm[7] e imm[6].

Los bits imm[4:1] y imm[11] de la instrucción B-type constituyen la porción inferior. La expansión los forma con {instr16[11:10], instr16[4:3], instr16[12]}, donde instr16[11:10] es offset[2:1], instr16[4:3] es offset[7:6], e instr16[12] es offset[5]. Este último bit se coloca en la posición imm[11], que es el bit inmediatamente anterior al bit de signo en el formato B-type.

Una vez que la instrucción B-type expandida llega a la etapa EX, el módulo `ex_stage.v` procesa la condición de branch. El fragmento relevante se encuentra en las líneas 38 a 47 de `ex_stage.v`:

```
wire eqE = (SrcAE == SrcBE);
wire ltE = ($signed(SrcAE) < $signed(SrcBE));
reg branchCond;
always @* case (funct3E)
  3'b000: branchCond = eqE;
  3'b001: branchCond = ~eqE;
  3'b100: branchCond = ltE;
  3'b101: branchCond = ~ltE;
  default: branchCond = 1'b0;
endcase
wire branchTakenE = BranchE & branchCond;
```

Para `funct3 = 000` (BEQ), la señal `branchCond` toma el valor de `eqE`, que es 1 cuando `SrcAE == SrcBE`. Dado que `SrcAE` contiene el valor del registro `rs1'` (con forwarding si es necesario) y `SrcBE` contiene el valor de `x0` (siempre cero), la condición se cumple exactamente cuando `rs1' == 0`. Si `branchCond` es 1 y `BranchE` está activa, la señal `PCSrcE` se establece en `2'b01` (como se define en la línea 49 de `ex_stage.v`), lo que hace que el multiplexor en `if_stage.v` seleccione `PCTargetE` como el próximo valor del PC.

El hazard unit, por su parte, trata esta instrucción como un branch convencional. No se requiere ninguna modificación, pues la señal `Branch` se activa en `maindec.v` para el opcode `1100011`, y el hazard unit utiliza `PCSrcTakenE` (definida en la línea 37 de `pipeline.v` como `PCSrcE != 2'b00`) para generar el flush en las etapas ID y EX cuando el branch es tomado. El cálculo de `PCTargetE` se realiza mediante el sumador `branch_adder` en la línea 36 de `ex_stage.v`, que suma el PC actual con el inmediato B-type extendido.

4.3.2 C.BNEZ (Branch if Not Equal to Zero comprimido)

La instrucción C.BNEZ es complementaria a C.BEQZ. Evalúa si el contenido de un registro del rango comprimido es distinto de cero y, en caso afirmativo, desvía el flujo a PC + offset. Su sintaxis es `c.bnez rs1', offset` y se expande a `bne rs1', x0, offset`. La codificación de 16 bits es idéntica en estructura a la de C.BEQZ, con la única diferencia de que el campo `funct3` toma el valor 111 en lugar de 110.

15–13	12	11–10	9–7	6–5	4–3	2	1–0
111	offset[5]	offset[2 :1]	rs1'	offset[4 :3]	offset[7 :6]	offset[8]	01

El código de expansión se encuentra en las líneas 92 a 99 del archivo `decompressor.v`.

```
3'b111: begin
    instr32 = {instr16[12],
               instr16[12], instr16[12], instr16[12],
               instr16[6], instr16[5], instr16[2],
               5'b00000, rs1_p, 3'b001,
               instr16[11:10], instr16[4:3], instr16[12],
               7'b1100011};
end
```

El lector observará que la única diferencia con respecto a C.BEQZ es el valor del campo `funct3`: 3'b001 en lugar de 3'b000. El resto de la expansión es idéntica, incluyendo la construcción del inmediato B-type y la fijación de `rs2` a `x0`. El valor `funct3 = 001` hace que en la etapa EX, la señal `branchCond` tome el valor `~eqE` (línea 43 de `ex_stage.v`), que es 1 cuando `SrcAE != SrcBE`. De esta manera, el

branch se toma si rs1' es distinto de cero.

4.4 Instrucciones de salto incondicional

La extensión RVC proporciona cuatro variantes de salto incondicional, distribuidas entre los cuadrantes Q1 y Q2. C.J y C.JAL pertenecen al cuadrante Q1 y se expanden a la instrucción JAL de RV32I, diferenciándose únicamente en el registro destino: x0 para C.J (descartando la dirección de retorno) y x1 para C.JAL (almacenando la dirección de retorno para llamadas a subrutina). C.JR y C.JALR pertenecen al cuadrante Q2 y se expanden a JALR, donde la dirección de salto se obtiene de un registro en lugar de un offset relativo al PC.

4.4.1 C.J (Jump comprimido)

La instrucción C.J transfiere el control incondicionalmente a la dirección PC + offset, sin almacenar dirección de retorno. Su sintaxis es c.j offset y se expande a jal x0, offset. El offset tiene 12 bits con signo y alineación a 2 bytes, lo que proporciona un rango de salto de aproximadamente ± 4 KB.

La codificación de 16 bits de C.J se presenta en la tabla siguiente.

15–13	12	11–10	9	8	7	6	5	4	3	2	1–0
101	offset[4]	offset[7:6]	offset[3]	offset[8]	offset[9]	offset[2]	offset[10]	offset[5]	offset[11]	offset[6]	01

El offset de 12 bits se encuentra distribuido en 11 campos individuales dentro del halfword, lo que refleja la naturaleza dispersa del formato J-type de RV32I, donde los bits del offset también aparecen en posiciones no contiguas.

El código de expansión se encuentra en las líneas 76 a 83 del archivo decompressor.v.


```

3'b101: begin
    instr32 = {instr16[12],
               instr16[8], instr16[10:9], instr16[6],
               instr16[7], instr16[2], instr16[11], instr16[5:3],
               instr16[12],
               {8{instr16[12]}},
               5'b00000, 7'b1101111};
end

```

La expansión produce una instrucción J-type con opcode 1101111. El registro rd se fija a 5'b00000 (x0), lo que significa que el valor de PC + 2 (la dirección de retorno) se descarta al escribirse en el registro cero. Esta es la semántica de un salto incondicional simple, a diferencia de una llamada a subrutina que sí preserva la dirección de retorno.

La construcción del inmediato J-type sigue el formato definido por RV32I, donde el offset de 21 bits (bits 0 a 20, con el bit 0 implícito en cero) se distribuye en: imm[20] en la posición 31, imm[10:1] en las posiciones 30 a 21, imm[11] en la posición 20, e imm[19:12] en las posiciones 19 a 12. La expansión reorganiza los 12 bits del offset RVC en estas cuatro porciones.

El bit instr16[12] aparece dos veces en la expansión: una vez como imm[11] (en la posición que corresponde en el formato J-type) y otra como bit de signo en la extensión {8{instr16[12]}}. Este último bloque produce los bits imm[19:12] mediante repetición del bit de signo, lo que reduce el rango efectivo del offset pero simplifica la expansión. Los bits instr16[8], instr16[10:9] e instr16[6] proporcionan imm[10:8] y imm[6]. El bit instr16[7] se coloca en imm[7]. El bit instr16[2] se coloca en imm[5]. Finalmente, el bit instr16[11] se coloca en imm[4] y los bits instr16[5:3] proporcionan imm[3:1].

Cuando la instrucción J-type expandida llega a la etapa EX, la señal Jump (generada por maindec.v en la línea 20) se activa:

```
7'b1101111: controls = 13'b1_011_0_0_10_0_00_1_0;
```

La señal Jump = 1 hace que PCSrcE se establezca en 2'b01 (línea 50 de ex_stage.v), seleccionando PCTargetE como el próximo PC. El valor de PCTargetE se calcula en la línea 36 de ex_stage.v sumando PCE (el PC de la instrucción en EX) con ImmExtE (el offset J-type extendido). El registro destino rd = x0 se escribe con PCPlus4E, pero como x0 es inmodificable, el resultado se descarta.

4.4.2 C.JAL (Jump and Link comprimido)

La instrucción C.JAL transfiere el control a PC + offset y, a diferencia de C.J, almacena la dirección de retorno (PC + 2) en el registro x1 (link register). Su sintaxis es c.jal offset y se expande a jal x1, offset. Es la versión comprimida de la instrucción jal de RV32I y se utiliza para llamadas a subrutina dentro del rango de ± 4 KB.

La codificación de 16 bits es idéntica a la de C.J en cuanto a la distribución del offset, pero con funct3 = 001 en lugar de 101.

15–13	12	11–10	9	8	7	6	5	4	3	2	1–0
001	offs et[4]	offs et[7 :6]	offs et[3]	offs et[8]	offs et[9]	offs et[2]	offs et[1 0]	offs et[5]	offs et[1 1]	offs et[6]	01

El código de expansión se encuentra en las líneas 35 a 42 del archivo decompressor.v.

```
3'b001: begin
```

```

instr32 = {instr16[12],
           instr16[8], instr16[10:9], instr16[6],
           instr16[7], instr16[2], instr16[11], instr16[5:3],
           instr16[12],
           {8{instr16[12]}},
           5'b00001, 7'b1101111};
end

```

La única diferencia con C.J es que el registro rd se fija a 5'b00001 (x1) en lugar de 5'b00000. Esto hace que la dirección de retorno PCPlus4F (que en este caso es PC + 2, dado que la instrucción es comprimida) se escriba en el registro x1. La etapa WB es la encargada de realizar esta escritura, tomando PCPlus4W como resultado cuando ResultSrc = 10.

El resto del comportamiento en el pipeline es idéntico al de C.J. La señal Jump = 1 desvía el PC hacia PCTargetE, y el hazard unit genera un flush en las etapas ID y EX para descartar las instrucciones que fueron cargadas antes de que se resolviera el salto.

4.4.3 C.JR (Jump Register comprimido)

La instrucción C.JR transfiere el control incondicionalmente a la dirección contenida en un registro, sin almacenar dirección de retorno. Su sintaxis es c.jr rs1 y se expande a jalr x0, rs1, 0. Esta instrucción permite saltar a direcciones calculadas dinámicamente, como las que se obtienen de un jump table o de un puntero a función.

Pertenece al cuadrante Q2 con funct3 = 100 y se distingue de otras instrucciones del mismo grupo porque instr[12] = 0 y instr[6:2] = 00000. La codificación de 16 bits se muestra en la tabla siguiente.

15–13	12	11–7	6–2	1–0
100	0	rs1	00000	10

El registro rs1 se codifica en 5 bits completos en las posiciones 11 a 7, lo que permite saltar a cualquier dirección almacenada en cualquier registro del banco. El campo rs2 en las posiciones 6 a 2 debe ser cero (00000) para que la instrucción sea reconocida como C.JR y no como C.MV. El código de expansión se encuentra en las líneas 114 a 116 del archivo decompressor.v.

```

if (!instr16[12]) begin
    if (instr16[6:2] == 5'b00000) begin
        instr32 = {12'b0, rd_f, 3'b000, 5'b00000, 7'b1100111};
    end else begin
        instr32 = {7'b0000000, rs2_f, 5'b00000, 3'b000, rd_f, 7'b0110011};
    end
end
end

```

La expansión produce una instrucción JALR con opcode 1100111. El inmediato se fija a 12 bits en cero (12'b0), lo que significa que no hay desplazamiento adicional: el salto es exactamente a la dirección contenida en rs1. El registro rs1 se obtiene del cable rd_f, definido como instr16[11:7]. Es importante notar que, aunque el cable se llama rd_f, en el contexto de C.JR este campo contiene el registro fuente rs1, no un registro destino. El nombre rd_f es genérico para el campo de 5 bits en las posiciones 11 a 7 del formato CR/CI, y su función semántica depende de la instrucción específica. El registro destino rd se fija a 5'b00000 (x0), descartando la dirección de retorno.

El módulo maindec.v decodifica el opcode 1100111 mediante la línea 21:

```
7'b1100111: controls = 13'b1_000_1_0_10_0_00_1_1;
```

Las señales relevantes son Jump = 1 y Jalr = 1. La combinación de ambas hace que en la etapa EX, la señal PCSrcE se establezca en 2'b10 (línea 49 de ex_stage.v), seleccionando PCJalrE como el próximo PC. El valor de PCJalrE se calcula en la línea 37 de ex_stage.v:

```
assign PCJalrE = {ALUResultE[31:1], 1'b0};
```

Donde ALUResultE es el resultado de sumar rs1 con el inmediato (que es cero). Esta operación de suma se realiza en la ALU con ALUControl = 0000 (ADD), y luego se fuerza el bit menos significativo a cero para garantizar la alineación a 2 bytes de la dirección de salto, un requisito de la especificación RISC-V para JALR.

4.4.4 C.JALR (Jump and Link Register comprimido)

La instrucción C.JALR transfiere el control a la dirección contenida en un registro y almacena la dirección de retorno (PC + 2) en el registro x1. Su sintaxis es c.jalr rs1 y se expande a jalr x1, rs1, 0. Es la versión comprimida de la instrucción jalr de RV32I y se utiliza para llamadas a subrutina indirectas, como en la invocación de funciones a través de punteros o en la implementación de retornos desde subrutinas (cuando rs1 = x1 o rs1 = x5).

Pertenece al mismo grupo que C.JR en el cuadrante Q2, pero se distingue porque instr[12] = 1 e instr[6:2] = 00000. La codificación de 16 bits se muestra en la tabla siguiente.

15–13	12	11–7	6–2	1–0
100	1	rs1	00000	10

El código de expansión se encuentra en las líneas 121 a 123 del archivo decompressor.v.

```
if (instr16[6:2] == 5'b00000) begin
    instr32 = {12'b0, rd_f, 3'b000, 5'b00001, 7'b1100111};
end
```

La expansión produce una instrucción JALR con opcode 1100111. La diferencia fundamental con C.JR es que el registro destino rd se fija a 5'b00001 (x1) en lugar de 5'b00000, lo que hace que la dirección de retorno PC + 2 se almacene en el link register. El registro fuente rs1 se obtiene de rd_f (instr16[11:7]) y el inmediato es cero.

Las señales de control generadas por maindec.v son las mismas que para C.JR: Jump = 1 y Jalr = 1. El comportamiento en el pipeline es idéntico, con la salvedad de que la etapa WB escribe PCPlus4W en x1 (registro x1) en lugar de en x0. Esta escritura permite que una instrucción posterior c.jr x1 (o c.jr x5, dependiendo de la convención de llamados) retorne de la subrutina utilizando la dirección almacenada.

Es importante destacar que la señal Jalr es la que distingue el comportamiento de JALR respecto de JAL en el cálculo del próximo PC. Mientras que JAL utiliza PCTargetE = PC + offset, JALR utiliza PCJalrE = (rs1 + 0) & ~1. Esta diferencia, codificada en las líneas 49 a 50 de ex_stage.v, permite que las instrucciones comprimidas de salto por registro funcionen correctamente sin ninguna modificación adicional en el datapath.

4.5 Cambios en el datapath para adaptación

Una vez analizadas las diez instrucciones comprimidas individualmente, es oportuno realizar una reflexión consolidada sobre los cambios que estas introducen en el datapath del pipeline. La conclusión principal es que ninguna de estas instrucciones requirió modificaciones en los módulos existentes del pipeline base, más allá de las que ya se introdujeron en la Parte 2 para habilitar el soporte RVC general (cambios en `imem.v`, `if_stage.v`, `pipeline.v` y `regfile.v`, documentados en la Sección 3.4 del informe anterior).

La razón de esta ausencia de cambios es que todas las instrucciones aquí tratadas se expanden a instrucciones del repertorio RV32I cuyos opcodes ya estaban implementados en la Parte 1 del proyecto. A continuación se presenta un mapeo explícito entre cada instrucción comprimida y el opcode RV32I al que se expande, junto con el módulo del pipeline que ya lo soportaba previamente.

Instrucción RVC	Expansión RV32I	Opcode	Módulo preexistente
C.LW	lw rd', offset(rs1')	0000011	maindec.v línea 15
C.SW	sw rs2', offset(rs1')	0100011	maindec.v línea 16
C.LWSP	lw rd, offset(x2)	0000011	maindec.v línea 15
C.SWSP	sw rs2, offset(x2)	0100011	maindec.v línea 16
C.BEQZ	beq rs1', x0, offset	1100011	maindec.v línea 18 + ex_stage.v línea 42

Instrucción RVC	Expansión RV32I	Opcode	Módulo preexistente
C.BNEZ	bne rs1', x0, offset	1100011	maindec.v línea 18 + ex_stage.v línea 43
C.J	jal x0, offset	1101111	maindec.v línea 20 + ex_stage.v línea 50
C.JAL	jal x1, offset	1101111	maindec.v línea 20 + ex_stage.v línea 50
C.JR	jalr x0, rs1, 0	1100111	maindec.v línea 21 + ex_stage.v líneas 37 y 49
C.JALR	jalr x1, rs1, 0	1100111	maindec.v línea 21 + ex_stage.v líneas 37 y 49

El módulo maindec.v contenía desde la Parte 1 las líneas de decodificación para los seis opcodes involucrados. Por ejemplo, la línea 15 (7'b0000011) genera las señales de control para las instrucciones de carga, que son compartidas por C.LW y C.LWSP. La línea 16 (7'b0100011) hace lo propio para las instrucciones de almacenamiento, compartidas por C.SW y C.SWSP. La línea 18 (7'b1100011) maneja las instrucciones branch, que incluyen C.BEQZ y C.BNEZ. Las líneas 20 y 21 (7'b1101111 y 7'b1100111) manejan los saltos incondicionales y por registro, respectivamente.

El módulo ex_stage.v también contenía desde la Parte 1 la lógica necesaria para evaluar las condiciones de branch (líneas 38 a 47) y para seleccionar el próximo

PC en función de las señales Branch, Jump y Jalr (líneas 49 a 50). Las condiciones eqE y ltE se calculan en las líneas 38 y 39 mediante comparadores de 32 bits, y el bloque case (funct3E) en las líneas 41 a 47 selecciona la condición adecuada. Para C.BEQZ, el funct3 = 000 activa eqE, mientras que para C.BNEZ, el funct3 = 001 activa $\sim$ eqE. Ninguna de estas condiciones requería soporte adicional, pues BEQ y BNE formaban parte del repertorio base de RV32I desde la primera entrega.

El cálculo de PCJalrE en la línea 37, necesario para C.JR y C.JALR, también estaba presente desde la Parte 1, pues la instrucción jalr de RV32I utiliza el mismo mecanismo independientemente de si proviene de una instrucción comprimida o estándar.

El módulo hazard_unit.v tampoco requirió modificaciones, tal como se explicó en la Sección 3.4.5 del informe anterior. El hazard unit opera sobre los números de registro de 5 bits de la instrucción expandida, sin conocimiento de si la instrucción original era de 16 o de 32 bits. Los mecanismos de forwarding, stalling y flushing funcionan de manera idéntica para todas las instrucciones aquí presentadas.

En resumen, la implementación de estas diez instrucciones demostró que la estrategia de expansión temprana en la etapa IF, mediante un descompresor combinacional, permite extender el repertorio de instrucciones de un pipeline RV32I sin modificar su núcleo de control ni su datapath. Todo el esfuerzo de implementación se concentró en el módulo decompressor.v, que actúa como un adaptador transparente entre el formato comprimido y el formato estándar de 32 bits.